

Core Concepts

Understanding the building blocks of dimensional modeling

Facts vs Dimensions Star vs Snowflake Design Process

Fact Tables

Fact tables store **quantitative measurements** of business processes. They contain the metrics that business users want to analyze.

Characteristics:

- Contain numeric, additive measures
- Foreign keys to dimension tables
- Large volume of records
- Sparse (many null values)

Example Measures:

Sales Amount

Quantity Sold

Profit

Discount

Dimension Tables

Dimension tables provide **descriptive context** for facts. They contain the "who, what, when, where, why, how" of business processes.

Characteristics:

- Contain textual, descriptive attributes
- Primary key referenced by fact tables
- Relatively small volume
- Dense (few null values)

Common Dimensions:

Date/Time

Product

Customer

Geography



COURSE NOTE

Interactive Exercise: Classify Attributes

Drag the attributes below into the correct category:

Available Attributes:

Facts (Measures)

Numeric, quantitative data

Sales Revenue

Quantity Ordered

Discount Amount

Dimensions (Attributes)

Descriptive, contextual data

Customer Name

Product Category

Store Location



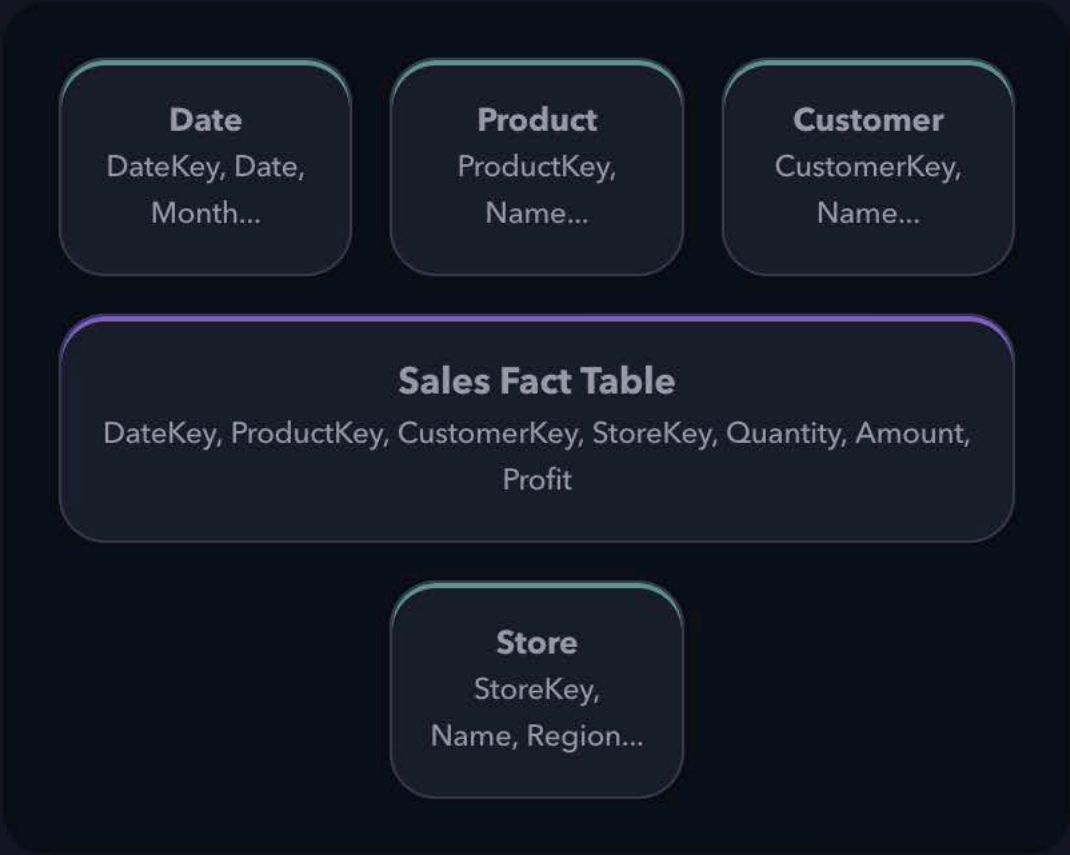
Excellent! You correctly classified all attributes.

Core Concepts

Understanding the building blocks of dimensional modeling

Facts vs Dimensions Star vs Snowflake Design Process

Star Schema



Benefits:

- ✔ Simple, intuitive structure
- ✔ Fast query performance
- ✔ Easy to understand and navigate
- ✔ Minimal joins required

Snowflake Schema



Benefits:

- ✔ Normalized structure
- ✔ Reduced storage space
- ✔ Better data integrity
- ✔ Easier maintenance

Core Concepts

Understanding the building blocks of dimensional modeling

Facts vs Dimensions Star vs Snowflake Design Process



COURSE NOTE

The Kimball Methodology

The Kimball methodology is a data warehousing design approach developed by Ralph Kimball that emphasizes creating user-friendly, performance-optimized dimensional models.



Business-Focused

Starts with understanding business processes and user needs



Atomic Grain

Emphasizes storing data at the most detailed level



Performance-Optimized

Uses star schema designs that optimize query performance



Iterative Process

Builds data warehouses incrementally, one process at a time



COURSE NOTE

The Four-Step Dimensional Design Process



Select Business Process

Choose the business process to model (e.g., Sales, Inventory, HR)



Declare the Grain

Define what each row in the fact table represents



Identify Dimensions

Determine the context surrounding the business process



Identify Facts

Select the numeric measurements that result from the business process

The Four-Step Process in Detail

Apply Kimball's methodology step by step

- 1 Step 1
- 2 Step 2
- 3 Step 3
- 4 Step 4

Step 1: Select the Business Process

The business process is the subject area you want to model. It should be:

- ✓

Atomic
Represent the most granular level of detail
- ✓

Business-focused
Align with how the business operates
- ✓

Measurable
Generate numeric facts that can be analyzed

COURSE NOTE

Explore Different Industry Scenarios

- 🏪

Retail Store
- 🏥

Healthcare
- 🏦

Banking
- 🏭

Manufacturing

Manufacturing: Production Line

Industry Focus:
Manufacturing operations and shop-floor execution

Modeled Process:
Manufacturing Production Events

Key Challenge:
Quality and efficiency optimization

Analytics Goal:
Production efficiency, quality control, supply chain coordination

WHY MODEL THIS PROCESS?

Production events determine throughput, quality, and cost on the shop floor. Modeling this process at an atomic level supports yield analysis, bottleneck identification, and end-to-end supply chain optimization.

The Four-Step Process in Detail

Apply Kimball's methodology step by step

- 1 Step 1
- 2 Step 2
- 3 Step 3
- 4 Step 4

Step 2: Declare the Grain

The grain defines what each row in your fact table represents. It must be:

Specific Clearly define what business event each row captures	Consistent All measures must be at the same level of detail	Atomic Choose the lowest level of detail available
---	---	--

Choose the "Grain"

Choose the appropriate grain for our retail sales process:

- ☐ **Daily Sales by Store**
One row per store per day
- ☒ **Individual Transaction Line Item**
One row per product per transaction
- ☐ **Monthly Sales Summary**
One row per month per product

✓ Correct! Individual transaction line items provide the most atomic level of detail, allowing for maximum analytical flexibility.

The Four-Step Process in Detail

Apply Kimball's methodology step by step

1 Step 1

2 Step 2

3 Step 3

4 Step 4

Step 3: Identify Dimensions

Dimensions provide the context for analysis. Ask these questions:

Who?

What people or organizations are involved?

Customer, Employee, Supplier

What?

What products or services are involved?

Product, Service, Promotion

When?

What time period does this cover?

Date, Time, Fiscal Period

Where?

What locations are relevant?

Store, Region, Territory

The Four-Step Process in Detail

Apply Kimball's methodology step by step

- 1 Step 1
- 2 Step 2
- 3 Step 3
- 4 Step 4

Step 4: Identify Facts

Facts are the numeric measurements that result from the business process.

Ask These Business Questions:

- **"What do we want to measure?"** - Revenue, quantity, costs, time
- **"What do stakeholders report on?"** - KPIs, metrics in dashboards
- **"What drives business decisions?"** - Profitability, performance indicators
- **"What gets counted or calculated?"** - Units sold, dollars earned, time spent

Good Facts Are:

- **Numeric:** Can be mathematically aggregated (sum, average, count)
- **Business-Relevant:** Meaningful to business users and decisions
- **Consistently Available:** Present in most/all business process events
- **Atomic:** At the same grain as your fact table

Fact Types

Additive Facts

Can be summed across all dimensions

Sales Amount

Quantity

Units

Semi-Additive Facts

Can be summed across some dimensions

Account Balance

Inventory Level

Non-Additive Facts

Cannot be meaningfully summed

Ratios

Percentages

Temperatures

Slowly Changing Dimensions (SCD) address how to handle changes to dimension attributes over time while preserving analytical capabilities and historical accuracy.

🎭 Real-World Scenario

Customer Sarah Johnson moves from New York to California. How do we handle this change in our Customer dimension while maintaining accurate historical sales reporting?

SCD Types

Type 0: Retain Original

No Change

Strategy: Never change the original value, even when source data changes

Example: Birth Date, Social Security Number, Original Account Opening Date

When to Use: Attributes that should never change

Type 1: Overwrite

Overwrite

Strategy: Update the attribute value, losing historical values

Example: Current phone number, email address, current job title

When to Use: When historical values are not needed

Type 2: Add New Record

Version

Strategy: Create a new dimension record with effective dates to track history

Example: Customer addresses, employee departments, product prices

When to Use: When historical accuracy is critical for analysis

Type 3: Add New Attribute

Previous/Current

Strategy: Keep both old and new values in separate columns

Example: Previous and current department, old and new product category

When to Use: Limited history needed, enable before/after analysis

Type 4: History Table

History Table

Strategy: Store detailed change history in a separate table while keeping the main dimension row current and lightweight

Example: Customer_History table with one row per version of a customer's address and segment

When to Use: When you need rich historical tracking but want to optimize the primary dimension for fast lookups

Late Arriving Dimensions occur when dimension records arrive after their corresponding fact records, creating referential integrity challenges in data warehouse loading processes.

🎭 Real-World Scenario

Your retail system processes sales transactions immediately, but new product information from suppliers arrives 24-48 hours later. How do you handle sales of products you don't yet have dimension records for?

Best Practices

- ✓ Always include 'Unknown' or 'Not Yet Available' dimension records
- ✓ Use consistent surrogate key patterns for default records
- ✓ Design ETL processes to handle out-of-order arrivals
- ✓ Implement proper error handling and alerting
- ✓ Establish SLAs with source systems for dimension data
- ✓ Create business rules for handling extended delays

Data Illustration: Fact vs Dimension Timing

Day 1 – Sales Fact Arrives First

TXNID	PRODUCTKEY	DATEKEY	QUANTITY	NE AM
500123	999999 (Unknown)	20231016	3	203

The fact row points to an **Unknown** product because the product master record has not arrived yet.

Day 2 – Product Dimension Arrives

PRODUCTKEY	SKU	NAME	CATEGORY
1004	SKU-1004	4K TV 55"	Electronics

An ETL reconciliation step later **repairs the foreign key** in the fact row, replacing 999999 with 1004 while keeping the original transaction intact.

Conformed Dimensions are shared across multiple fact tables with identical structure and content, enabling consistent analysis across different business processes.

 Real-World Scenario

A retail company wants to analyze both in-store and online sales data. How can they ensure consistent customer and product analysis across these different sales channels?

Benefits

- ✓ Enterprise-Wide Analysis: Compare metrics across different business processes
- ✓ Drill-Across Capability: Query multiple fact tables simultaneously
- ✓ Consistent Reporting: Same dimension values across all reports
- ✓ Reduced Development: Build once, use many times
- ✓ Improved Governance: Single source of truth for dimension data

Data Illustration: Shared Customer Dimension

Customer Dimension (Conformed)

CUSTOMERKEY (PK)	CUSTOMERID	NAME	SEGMENT
1001	CUST-1234	Alex Rivera	Premium

Online Sales Fact

SALEKEY (PK)	CUSTOMERKEY (FK)	PRODUCTKEY (FK)	DATEKEY (FK)	SALESAMOUNT
90001	1001	2005	20231016	199.99

In-Store Sales Fact

SALEKEY (PK)	CUSTOMERKEY (FK)	PRODUCTKEY (FK)	STOREKEY (FK)	DATEKEY (FK)
91001	1001	2005	3002	20231017

CustomerKey 1001 represents the same person in both fact tables, enabling true drill-across analysis.

Junk Dimensions consolidate multiple low-cardinality flags, indicators, and attributes into a single dimension table to avoid dimension explosion while maintaining analytical flexibility.

Real-World Scenario

An e-commerce company has numerous order attributes like payment method, gift wrap status, shipping priority, and promotion codes. How can they efficiently model these attributes without creating dozens of tiny dimensions?

Benefits

- ✓ Simplified Fact Table: Reduces the number of foreign keys
- ✓ Improved Query Performance: Fewer joins needed for analysis
- ✓ Reduced ETL Complexity: Simpler loading process
- ✓ Efficient Storage: Pre-combines common attribute combinations

Implementation Considerations

- Cardinality Management: Number of rows equals Cartesian product of all values
- Only for Low-Cardinality: Best for attributes with few distinct values
- Pre-populate All Combinations: Generate all possible combinations in advance

Bridge tables solve many-to-many relationships between facts and dimensions, enabling complex analytical scenarios while maintaining dimensional model integrity.

 Real-World Scenario

A bank wants to analyze account activity by household, but customers can belong to multiple households (e.g., personal account + joint account with spouse + business account with partner).

Examples

Customer-Household Bridge

Enables: Show total household assets across all family members

Product Bundle Bridge

Enables: Analyze sales of meal bundles vs individual items

Employee Department Bridge

Enables: Calculate costs for employees working across multiple departments

Data Illustration: Bridge Table Structures

Customer_Household_Bridge

CUSTOMERKEY (FK)	HOUSEHOLDKEY (FK)	RELATIONSHIPTYPE	ALLOCATIONFACTOR
2001	5001	Primary	0.6
2002	5001	Spouse	0.4

Allocation factors sum to **1.0** per household to prevent double-counting.

Product_Bundle_Bridge

BUNDLEKEY (FK)	PRODUCTKEY (FK)	QUANTITY	ALLOCATIONFACTOR
7001	3001	1	0.5
7001	3002	2	0.5

Bundle revenue can be allocated back to individual products.

Employee_Department_Bridge

EMPLOYEEKEY (FK)	DEPARTMENTKEY (FK)	ROLE	TIMEALLOCATION
8001	4001	Analyst	0.5
8001	4002	Analyst	0.5

Labor costs can be distributed across multiple departments based on **TimeAllocation**.

 Best Practices

Allocation Factors: Use factors that sum to 1.0 to prevent double-counting

Weighting Strategy: Choose based on business rules (equal split, revenue-based, time-based)

Temporal Handling: Include effective dates when relationships change over time

Query Patterns: Design for common query patterns (drill-across, roll-up)